



Silicon Info

A floating CPU, GPU and Neural Engine widget for Apple Silicon Macs

Documentation, version 1.0

Copyright © 2026 Ray Munro. All rights reserved.

1. Overview

Silicon Info is a small always-visible panel for the desktop. It breaks Mac activity into the pieces that make up an Apple Silicon chip instead of showing one combined number. The panel has four cards:

- **CPU** performance cores and efficiency cores, shown separately.
- **GPU** device utilization with render and tiler load.
- **Neural Engine** power draw and activity history.
- **Power** how total power divides between the three blocks.

Each card is a summary. Click a card to expand it into a detailed view, and use the Back control to return.

2. Requirements

Item	Requirement
Hardware	Mac with Apple Silicon (developed on an M4 Max)
Operating system	macOS 14 or later
Build tools	Xcode command line tools with Swift 5.9 or later
Privileges	An administrator password, entered once per launch, for power and frequency data

3. Installation and use

Build

```
./build.sh
```

This compiles a release build, draws the app icon, assembles `Silicon Info.app` and signs it ad hoc for local use.

Run

```
open "Silicon Info.app"
```

The widget appears at the top right of the main display. It has no Dock icon or menu bar item. Drag it anywhere by its background. Its position is remembered, and the window is kept fully on screen when a card expands.

Quit

```
pkill SiliconWidget
```

4. The views

Overview

Card	Shows
CPU	One bar per core, grouped into performance cores (blue) and efficiency cores (green), each group's average load, package power and a load history line.
GPU	A ring for device utilization, render and tiler bars, power and a history line.
Neural Engine	A ring for power draw and a history line.
Power	A single bar divided between CPU, GPU and Neural Engine, with the total in watts.

CPU detail

Section	Contents
Clusters	For the efficiency cluster and each performance cluster: current frequency, active share, and a histogram of time spent at each clock step.
Cores	Every core with a stacked bar of user time (blue or green) and system time (red), its current clock speed and total load.
Top processes	The busiest processes, as a percentage of one core.
Power	Two minutes of CPU power with average and peak.

GPU detail

Section	Contents
Overview	Utilization ring, current frequency, maximum clock, active and idle share, power.
Frequency residency	Time spent at each GPU clock step.
Engines	Device, render and tiler utilization.
Memory	GPU memory in use and allocated. This memory is shared with the system.
Top GPU processes	Processes using GPU time, as a share of the GPU.

Neural Engine detail

Shows current power, average and peak over two minutes, energy used since launch, an active or idle state and a power history chart. Load is power as a share of an assumed 8 W ceiling, so treat the ring as relative.

Power detail

Shows the live split between CPU, GPU and Neural Engine, a stacked two-minute history where the top edge is the total, and average power, peak power and energy in milliwatt-hours for each block since launch.

5. Data sources

Data	Source	Needs admin
Per-core load, user and system split	Mach <code>host_processor_info</code>	No
GPU utilization, render, tiler, memory	IOKit accelerator statistics	No
CPU, GPU and Neural Engine power	<code>powermetrics</code>	Yes
Core and cluster frequency, residency	<code>powermetrics</code>	Yes
GPU frequency and residency	<code>powermetrics</code>	Yes
Per-process CPU and GPU time	<code>powermetrics</code> tasks sampler	Yes

Core order follows Apple Silicon convention: efficiency cores are numbered first, then performance cores.

6. Privileges and safety

Reading power and frequency needs root access. On launch the widget shows the standard macOS administrator dialog. After approval, a small shell loop runs as root. It does only one thing: it runs `powermetrics` once per second and writes the text output to `/private/tmp/siliconwidget-powermetrics.txt`, which the widget reads.

- The loop exits on its own about a second after the last Silicon Info process quits.
- Relaunching quickly reuses the running loop and does not ask again.
- Nothing is sent over the network. The output file contains only power, frequency and process names with CPU and GPU time.
- If you cancel the dialog, the widget keeps running and shows `n/a` for the values that need it.

The output file is readable by other local users, so process names on the machine are visible to them while the widget runs.

7. Limitations

macOS does not expose the following on this hardware, so they are not shown:

- **Neural Engine utilization, frequency or per-process use.** Only power draw is available.
- **Per-cluster CPU power.** Only the CPU total is reported.
- **DRAM bandwidth and media engine power.** These are not readable on this chip.

Per-process GPU time is often zero because few processes keep the GPU busy at any moment.

8. Troubleshooting

Symptom	Cause and fix
Power shows <code>n/a</code>	The administrator dialog was cancelled. Quit and relaunch, then approve it.
Power graphs are empty after launch	History fills at one sample per second. Wait a minute.
Neural Engine reads 0 W	The engine is idle. Run something that uses it, such as Core ML work, to see it move.
Widget is off screen	Delete the saved position with <code>defaults delete com.raymunro.siliconinfo</code> and relaunch.
Build fails	Install the command line tools with <code>xcode-select --install</code> .

9. Project layout

Path	Purpose
<code>Sources/SiliconWidget/main.swift</code>	Creates the borderless floating window and keeps it anchored and on screen.
<code>Sources/SiliconWidget/Sampler.swift</code>	Samples core load and GPU statistics each second and holds the history.
<code>Sources/SiliconWidget/PowerFeed.swift</code>	Starts the privileged loop and parses the <code>powermetrics</code> output.
<code>Sources/SiliconWidget/Views.swift</code>	The overview cards and expand and collapse behavior.
<code>Sources/SiliconWidget/Detail.swift</code>	The CPU, GPU, Neural Engine and Power detail views.
<code>scripts/make_icon.swift</code>	Draws the app icon.
<code>build.sh</code>	Builds and signs the app bundle.